Implementing Redis Caching in Real ag-kit EntityManager

A concrete, file-by-file guide — based on the working demo at entitymanager-caching-demo, checked against real ag-kit source

Contents

1. Current state of real ag-kit (confirmed by reading the source)

Before writing a single line, here's exactly what exists today in `ag-kit` — verified by reading the actual files, not assumed:

Area	Current state
`Shared/Infrastructure/Ag.Cache/`	Exists, but is an **empty scaffold** — just the SDK-template `Class1.cs` and a `.csproj` with zero package references. None of the real caching code exists here yet.
Redis anywhere in the solution	**Not present at all.** No `StackExchange.Redis` package reference anywhere, no `"Redis"` section in any `appsettings.json`, no Redis connection string.
`Host/EntityManager.Api/Program.cs`	Thin: `AddAgConfigure` → `AddSwaggerGen` → `AddEntityManagerModule` → build → `UseAg` / `UseSwagger` / `UseSwaggerUI` → `MapEntityManagerEndpoints`. No cache-related call anywhere.
`EntityManagerModuleServiceExtensions.cs`	`AddMediator` is called with `ServiceLifetime.Scoped` and one marker assembly (`EntityManager.Application.Commands.Agent.GetAgentCommand`). **Zero `IPipelineBehavior<,>` registrations exist today** — this is a clean, empty insertion point.
Mediator package version	`Mediator.Abstractions` / `Mediator.SourceGenerator` **3.0.2** — matches the demo exactly. No version-compatibility work needed.
`GetAgentQuery.cs` (real)	`public record GetAgentQuery(int? AgentKey, string? SeoName, string? ClientCode) : IRequest<JsonElement?>;` — does **not**

	implement `ICacheableQuery` (that interface doesn't exist in real ag-kit yet). Handler logic is otherwise identical to the demo's copy.
`AgentRepository.GetAgentList` vs `GetAgentDetail`	**Confirmed real, pre-existing inconsistency** (see section 5) — `GetAgentDetail` filters `IsDisplayedOnWebsite` + inner-joins Office/Company; `GetAgentList` filters neither.

2. The request flow — exactly where caching plugs in

This answers "from which section does it hit the cache pipeline" directly: caching is **not** tied to any specific layer's file — it's a Mediator pipeline behavior, registered once in DI as an **open generic** (`IPipelineBehavior<,>`). Mediator automatically runs it in front of the real handler for *every single request* sent through `sender.Send(...)` , regardless of which module or which Presentation endpoint triggered it.

```
HTTP GET /Agent/get?agentKey=131&clientCode=WRE
   │
   ▼  [Presentation layer]  EntityManager.Presentation/Endpoints/AgentEndpoints.cs
   │     app.MapGet("/Agent/get", async (ISender sender, int? agentKey, ...) => {
   │         var result = await sender.Send(new GetAgentQuery(agentKey, seoName, clientCode));
   │     });
   ▼
   [Mediator's internal dispatch — generated code, not hand-written]
   │     runs every IPipelineBehavior<GetAgentQuery, JsonElement?> registered in DI, in order,
   │     wrapping the real handler like a set of middleware
   ▼
   [Infrastructure layer, but NOT tied to EntityManager specifically]
   │     Ag.Cache/CachingPipelineBehavior<TRequest,TResponse>.Handle(request, next, ct)
   │        - is request an ICacheableQuery?  (GetAgentQuery now says yes)
   │        - BuildCacheKey() -> "ety:WRE:agent:get:agentkey=131"
   │        - Redis GET that key
   │        - HIT  -> return immediately, done (no DB call at all)
   │        - MISS -> call next(request, ct)  ──────────────┐
   ▼                                                        │
   [Application layer]  EntityManager.Application/Queries/  │
      GetAgentQueryHandler.Handle(request, ct)  ◄───────────┘
         - calls agentRepository.GetAgentDetail(...)
   ▼
   [Infrastructure layer]  EntityManager.Infrastructure/Persistance/Repositories/AgentRepository.cs
      -> EF Core -> real idc_ety SQL Server
   ▼
   result bubbles back up through CachingPipelineBehavior, which STOREs it in Redis with the query's Ttl,
   then returns it to the Presentation endpoint -> HTTP response
```

> **Important scope note:** because the pipeline behavior is registered as an **open generic singleton** (`services.AddSingleton(typeof(IPipelineBehavior<,>), typeof(CachingPipelineBehavior<,>))`), it applies to *every* Mediator request resolved from that same service collection — not just Agent/Office/Company queries. This is

> safe by construction: the very first line of `Handle()` is `if (request is not ICacheableQuery cacheable) return await next(...)`, so every write `Command` in EntityManager (and, if this same registration pattern is reused in another module's host later, every request there too) passes straight through untouched, at the cost of one cheap type check. Nothing needs caching opted out — only queries that explicitly implement `ICacheableQuery` are ever touched.

3. Step-by-step implementation

1 Build out the real Ag.Cache project

`Shared/Infrastructure/Ag.Cache/Ag.Cache.csproj` today has no package references. Add the same five this demo uses (all already used elsewhere in the ag-kit solution, so no new approvals needed for the Mediator one):

```
<ItemGroup>
  <PackageReference Include="StackExchange.Redis" Version="2.8.24" />
  <PackageReference Include="Mediator.Abstractions" Version="3.0.2" />
  <PackageReference Include="Microsoft.Extensions.Configuration.Abstractions" Version="10.0.0" />
  <PackageReference Include="Microsoft.Extensions.Configuration.Binder" Version="10.0.0" />
  <PackageReference Include="Microsoft.Extensions.DependencyInjection.Abstractions" Version="10.0.0" />
</ItemGroup>
```

Delete the placeholder `Class1.cs`, and add these six files — copied essentially unchanged from `entitymanager-caching-demo/EntityManager/Shared/Infrastructure/Ag.Cache/` (see the companion PDF, "Ag.Cache — Full Folder Details", for the complete content and reasoning behind each):

- `ICacheableQuery.cs` — the opt-in interface
- `CacheKeyBuilder.cs` — `Build()` and `BuildFromObject()`
- `CacheKeyIgnoreAttribute.cs` — reflection opt-out marker
- `CachingPipelineBehavior.cs` — the actual caching logic
- `AgCachingServiceExtensions.cs` — startup wiring

None of these five files reference anything EntityManager-specific — this project has zero knowledge of Agent/Office/Company. It can be dropped into `Shared/Infrastructure` and reused by any other module later without modification.

2 Add Redis config to appsettings.json

Real ag-kit's `Host/EntityManager.Api/appsettings.json` has no `"Redis"` section today — add one alongside the existing `ConnectionStrings` / `Jwt` / `CDN` sections:

```
"Redis": {
  "Host": "<your Redis host>",
  "Port": "6379",
  "Password": "<set via environment/secret, never checked in>",
```

```
  "Ssl": false
}
```

> Whatever real Redis instance backs this in production/staging, treat its password exactly like the database credentials — environment variable or secret store, never committed to `appsettings.json` directly, for the same reasons already applied to `IDC_ETY` 's connection string in the demo.

3 Wire `AddAgCaching` into Program.cs

One line added to the real `Host/EntityManager.Api/Program.cs` :

```
using Ag.Cache;                                                    // add
using EntityManager.Infrastructure.DependencyInjection;
using EntityManager.Presentation;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddAgConfigure(builder.Configuration);
builder.Services.AddAgCaching(builder.Configuration);              // add - connects to Redis
builder.Services.AddSwaggerGen();
builder.Services.AddEntityManagerModule(builder.Configuration);

var app = builder.Build();

app.UseAg();
app.UseSwagger();
app.UseSwaggerUI();
app.MapEntityManagerEndpoints();

app.Run();
```

4 Register the pipeline behavior

Inside `EntityManagerModuleServiceExtensions.cs` , right after the existing `AddMediator(...)` call, add the same registration the demo uses and has verified working end-to-end:

```
services.AddMediator(config =>
{
    config.ServiceLifetime = ServiceLifetime.Scoped;
    config.Assemblies = [typeof(EntityManager.Application.Commands.Agent.GetAgentCommand)];
});

// add - Mediator will now run CachingPipelineBehavior before every query's real handler
services.AddSingleton(typeof(IPipelineBehavior<,>), typeof(CachingPipelineBehavior<,>));

services.AddScoped<IAgentRepository,AgentRepository>();
services.AddScoped<IOfficeRepository,OfficeRepository>();
services.AddScoped<ICompanyRepository,CompanyRepository>();
```

> This exact registration line (`services.AddSingleton(typeof(IPipelineBehavior<,>), typeof(CachingPipelineBehavior<,>))`) is what the demo actually uses and has confirmed working — it's a direct DI registration, not something routed through a Mediator-specific config API, so it doesn't depend on any Mediator builder method beyond what real ag-kit's `AddMediator` call already sets up.

5 Add `ICacheableQuery` to each Get/List query

For each of the 3 entities' single-record Get query and List query (6 records total: `GetAgentQuery` , `GetOfficeQuery` , `GetCompanyQuery` , `GetAgentListQuery` , `GetOfficeListQuery` , `GetCompanyListQuery`), the change is the same shape every time. Using the real, current `GetAgentQuery.cs` as the concrete example:

Today (real ag-kit)	After
`public record GetAgentQuery(int? AgentKey, string? SeoName, string? ClientCode) : IRequest<JsonElement?>;`	`public record GetAgentQuery(int? AgentKey, string? SeoName, string? ClientCode) : IRequest<JsonElement?>, ICacheableQuery { ... }`

```
public record GetAgentQuery(int? AgentKey, string? SeoName, string? ClientCode)
    : IRequest<JsonElement?>, ICacheableQuery
{
    public string? BuildCacheKey() => CacheKeyBuilder.BuildFromObject(
        ClientCode, "agent", "get", this, nameof(ClientCode));

    public TimeSpan Ttl => TimeSpan.FromMinutes(60);        // pick real production values here, not the demo's flat 10 min
    public TimeSpan RefreshWindow => TimeSpan.FromMinutes(10);
};

public class GetAgentQueryHandler : IRequestHandler<GetAgentQuery, JsonElement?>
{
    // unchanged - the real business logic in the handler needs ZERO modification
}
```

> **Record-shape inconsistency to handle:** real `GetCompanyQuery` / `GetOfficeQuery` use a required non-nullable `int CompanyKey` / `int OfficeKey` (they throw if `<= 0`), while `GetAgentQuery` uses a nullable `int? AgentKey` combined with `SeoName` . This doesn't break `BuildFromObject()` — reflection works the same either way — but it does mean `ClientCode` vs `ClientID` naming is inconsistent across the 6 records (confirmed: `GetAgentListQuery` / `GetCompanyListQuery` / `GetOfficeListQuery` all use `ClientID` , while the single-Get queries use `ClientCode`) — make sure each query's `BuildCacheKey()` excludes whichever of the two names that specific record actually has via `nameof(...)` , rather than copy-pasting `nameof(ClientCode)` everywhere.

> Pick real `Ttl` / `RefreshWindow` values deliberately per query type rather than copying the demo's flat 10-minute default for everything — the demo used one flat value everywhere purely to make manual testing predictable. A sensible starting point echoing typical production tuning: longer TTL for single-record Gets (rarely change), shorter TTL for Lists (harder to invalidate precisely, so lean on a shorter natural expiry instead).

4. The missing piece: invalidation needs a watcher

Everything above makes reads fast (cache-aside) but does **nothing about staleness** on its own — a cached value only disappears when its TTL runs out, unless something actively deletes the key the moment the underlying row changes. In the demo, that "something" is `Tools/SandboxWatcher`, which only works because the sandbox is a private LocalDB copy this demo's own test endpoints write to.

> **Real ag-kit has no equivalent today, and needs one designed — this isn't a copy-paste step.** Real `idc_ety` is written to by some other real system (the CMS/admin tooling), not by EntityManager.Api itself (EntityManager.Api is GET-only, matching real DB permissions). Whatever writes to `idc_ety` is the thing that needs to trigger invalidation — EntityManager doesn't control it.

Two realistic options, in order of how close they are to what the demo already proves works:

Option	How it would work	Trade-off
A. Polling watcher (same technique as `SandboxWatcher`)	A small standalone background service polls `idc_ety` every few seconds for rows where `LASTMODIFIED > last checkpoint`, and for each one, calls `CacheKeyBuilder.Build(...)` with that row's own `ClientCode` +business key to reconstruct the exact Redis key, then deletes it (optionally refilling it immediately via an internal refresh endpoint, exactly like the demo does).	Simple, proven (this demo validates the whole mechanism against real query shapes), but polling — some unavoidable delay between the real write and invalidation, and a continuous background query load on `idc_ety`.
B. Change Data Capture / eventing	If the team that owns the write side of `idc_ety` can publish a change event (SQL Server CDC, a message bus, a webhook) whenever a row changes, a listener consumes that event and invalidates the matching key immediately instead of polling.	Lower latency, no continuous polling load — but requires buy-in and work from whoever owns the write path, which is outside EntityManager's control.

Whichever option is chosen, it also needs the second half already proven in the demo: a consumer that drains `ety:refresh:queue` (the stream `CachingPipelineBehavior` pushes onto on near-expiry HITs) and proactively refreshes those keys — otherwise near-expiry entries in real production just sit unread and expire normally, same as if the Stale-While-Revalidate half of the pattern didn't exist. The refresh logic itself needs no new code: it's the same "reconstruct the query from the key, run the real handler, write the result back to Redis" approach as `InternalCacheEndpoints.cs` in the demo — that endpoint (or an equivalent) needs to exist in real ag-kit too, for the watcher to call.

5. Known risks and a pre-existing issue worth fixing alongside this

> **Confirmed real, pre-existing inconsistency** in `AgentRepository.cs` — `GetAgentDetail` (backing the single `/Agent/get`) filters `IsDisplayedOnWebsite = true` and inner-joins Office+Company (excluding an agent entirely if either join fails). `GetAgentList` (backing `/Agent/list`) filters neither — it queries the flat `Agent` table with no `IsDisplayedOnWebsite` check and no join at all.

This matters specifically *because* of caching: today, without caching, this inconsistency just means a list result can include an agent whose individual page 404s. Once caching is introduced, the same inconsistency becomes an invalidation-precision problem too — a list-cache entry could keep referencing an agent that a real production write later excludes from individual lookup, and nothing about the caching layer itself would flag this as wrong. The demo fixed this in its own `AgentRepository.GetAgentList()` by adding the same `IsDisplayedOnWebsite` filter and the same existence-check joins `GetAgentDetail` already applies — worth doing the same in real ag-kit before or alongside rolling out caching, not after.

> **Recommended rollout order:** (1) build out `Ag.Cache` and wire the pipeline behavior with zero queries opted in yet — this is a safe, no-behavior-change deploy, since `ICacheableQuery` won't exist on anything. (2) Opt in the single-record Get queries first (lowest invalidation complexity — one row, one key). (3) Fix the `AgentRepository.GetAgentList` inconsistency. (4) Opt in the List queries. (5) Build and deploy the watcher (Option A or B above) — caching without invalidation is not safe to leave running unattended for long in production, even with a short TTL as a backstop.

Based on the working demo at `entitymanager-caching-demo/EntityManager` and real ag-kit source read directly from `d:\AG\CM\ag-kit` at the time of writing. Re-verify file paths and current registrations against ag-kit's actual state before implementing, in case the codebase has moved on since.